

# Lesson 8: Deploying a Full-Stack React App to Production

## Mission Brief

Your React and Next.js work has only ever lived on your own machine. A real developer's portfolio needs a live URL that anyone — a recruiter, a friend, a fellow Lumexa cadet — can visit from anywhere in the world. Today you take your Vite React apps and your Next.js portfolio to production: building them correctly, catching build errors before they become live bugs, and deploying them with Vercel.

## Learning Objectives

By the end of this lesson, you will be able to:

- Explain the difference between a dev server and a production build
- Run and interpret a production build ( `npm run build` ) for both Vite and Next.js projects
- Fix common build-time errors (unused imports, missing keys, type mismatches)
- Deploy a Vite app and a Next.js app to Vercel
- Set an environment variable for a deployed site

## What You'll Build

Production builds of all three of your portfolio projects, verified locally, then deployed live to Vercel with working public URLs.

## Prerequisites

- Completed Projects 07, 08, and 09 (or equivalent React/Next.js apps)
- A GitHub account (recommended) and a Vercel account
- Comfortable running terminal commands ( `npm install` , `npm run <script>` )

## Key Concepts

- Dev server vs. production build (optimization, minification, bundling)
- `npm run build` and reading build output/errors
- Common build failures and how to fix them
- Deploying a static Vite app vs. a server-rendered Next.js app
- Environment variables in a deployed app

## Concept Explanation

Everything you've built so far ran through `npm run dev` — a **development server** optimized for fast feedback: instant reloads, unminified code, and helpful (verbose) error overlays. None of that is what you want to actually ship to real visitors — dev builds are larger, slower, and often include extra debugging code. **Production builds**, created with `npm run build`, instead bundle and minify your code, strip out development-only warnings, optimize images and assets, and produce the smallest, fastest version of your app.

For a Vite project (Projects 07 and 08), running `npm run build` produces a `dist/` folder containing static HTML, CSS, and JS files ready to be hosted anywhere:

```
npm install
npm run build
```

If the build succeeds, you'll see output listing the generated files and their sizes. If it fails, Vite prints the exact file and line where something went wrong — commonly a typo'd import path, a variable used but never defined, or (if using a linter) a rule violation.

For a Next.js project (Project 09), `npm run build` does more: it also pre-renders pages, checks types (if using TypeScript), and reports which routes are static vs. dynamic:

```
npm install
npm run build
```

A clean build prints a route summary table (something like `○ (Static) / ○ (Static) /about • (SSG) /projects/[slug]`). Any error here — a missing `export default`, a hook used without `"use client"`, an import from a file that doesn't exist — must be fixed before deploying, because a broken production build cannot be deployed at all.

Common build-time errors and their fixes:

- **"X is not defined"** — an import was forgotten or misspelled; check the top of the file.
- **"You're importing a component that needs useState... only works in a Client Component"** — add `"use client"` as the literal first line of that file.
- **Unused variable/import warnings (sometimes elevated to errors by strict linting)** — delete the unused import, or actually use it.
- **"Module not found"** — a relative import path (`./component` vs `./Component`) doesn't match the real filename on disk, which matters especially on case-sensitive deployment servers even if your own OS is case-insensitive.
- **Missing key prop warnings** in a list — while usually just a warning, fix these anyway; they can cause real bugs with reordered lists (Lesson 2).

Once a build succeeds locally, deploying to **Vercel** (built by the creators of Next.js, and equally good for static Vite apps) is typically:

1. Push your project to a GitHub repository.
2. In Vercel, "Import Project" and select that repository.
3. Vercel auto-detects the framework (Vite or Next.js) and sets the correct build command ( `npm run build` ) and output directory ( `dist` for Vite, automatic for Next.js).
4. Click Deploy — Vercel runs the exact same `npm install && npm run build` in its own environment, and if it succeeds, assigns your app a live `https://your-project.vercel.app` URL.

If your app needs a secret value (an API key, for instance) that shouldn't be committed to your code, you set it as an **environment variable** in Vercel's project settings rather than hardcoding it — locally this is mirrored with a `.env.local` file (which should always be listed in `.gitignore` so secrets never get committed to version control).

## Guided Walkthrough

1. In Project 07's folder, run `npm install` followed by `npm run build` ; confirm it completes with no errors and produces a `dist/` folder.
2. Repeat for Project 08.
3. In Project 09's folder, run `npm install` followed by `npm run build` ; read the route summary table it prints and confirm every page you built (Home, About, Projects, the dynamic project pages) appears in it.
4. Deliberately introduce a build error — comment out an import that's still used elsewhere in the file — run the build again, read the exact error message and file/line it points to, then fix it and confirm the build passes again.
5. (If you have GitHub/Vercel accounts) Push one project to GitHub and deploy it through Vercel's dashboard, then visit the live URL it gives you and confirm the deployed site matches what you saw locally.

## Common Bugs & Debugging Tips

- A build that works with `npm run dev` but fails with `npm run build` usually means dev mode was silently tolerating something (like a missing `"use client"` that only matters once code is pre-rendered) — always run a real build before considering a project "done."
- If `npm install` itself fails, check your Node.js version against the project's `package.json` "engines" field, if present, and try deleting `node_modules` and `package-lock.json` and reinstalling clean.
- Deploying and seeing a blank page (but the build succeeded locally) is very often an incorrect base path or a missing environment variable that only exists locally in `.env.local` — check the hosting platform's environment variable settings.
- If images don't load in production but did in dev, confirm they're either imported properly as modules or placed in the framework's designated public/static assets folder, and referenced with the correct relative path.

## Check Your Understanding

1. What is the practical difference between `npm run dev` and `npm run build` ?

2. Why can a project run fine in dev mode but fail `npm run build` ?
3. What file/folder does a Vite production build produce, and what does it contain?
4. Why should secrets (API keys) go into environment variables instead of being hardcoded in your source code?
5. What must be true about a production build before you can deploy it?

## Mini Quiz + Answer Key

**Q1.** What does `npm run build` produce that `npm run dev` does not? A) A live-reloading dev server B) An optimized, minified production bundle C) Nothing different D) A database

**Q2.** A component using `useState` fails only during `npm run build`, not `npm run dev`. What's the most likely cause? A) A missing `"use client"` directive in a Next.js App Router file B) A CSS typo C) Too many comments D) The dev server is broken

**Q3.** Where should a secret API key live for a deployed app? A) Hardcoded directly in a component file B) As an environment variable set in the hosting platform (and `.env.local` locally, gitignored) C) In a public GitHub README D) In the `dist/` folder

**Q4.** What must be true before you can successfully deploy a project to Vercel? A) Nothing, Vercel fixes errors automatically B) `npm run build` must complete successfully with no errors C) The project must use Next.js specifically D) You must delete all tests first

**Answer Key:** 1-B, 2-A, 3-B, 4-B

## Lesson Recap

You learned why production builds differ from dev servers, how to run and read `npm run build` output for both Vite and Next.js projects, how to diagnose and fix the most common build-time errors, and how to take a passing build live on Vercel with proper environment variable handling for secrets.

## Homework

Run `npm run build` on all three of your portfolio projects from a completely clean `node_modules` (delete it and `package-lock.json`, then `npm install` fresh) and confirm all three still build with zero errors — this "clean install" check is exactly what a real deployment platform does, and it's the best way to be sure a project isn't secretly relying on stale local state.

## Portfolio Project Connection

This lesson's build-and-deploy process is the final step for Projects 07, 08, and 09 — the same `npm install && npm run build` verification you'll run to confirm every portfolio project genuinely works, with no placeholders, before calling any of them finished.